

3장 컴퓨터 그래픽스의 기본 요소

3-1. 점 그리기

동차좌표

$\{x, y, z, h\}$ 로 h 는 동차 파라미터로 3차원 데카르트 좌표의 경우 $x = hx, y = hy, z = hz$ 이다.

2차원 좌표로 축소하면 $(1, 2, 1), (2, 4, 2), (3, 6, 3)$ 이라는 좌표가 있다. 이들은 일직선 상에 존재하는 점들로, 이에 대한 데카르트 좌표는 이 점들을 원점을 향해 투영한 투영선이 $h = 1$ 인 평면과 교차하는 좌표 $(1, 2)$ 이다.

함수

- 3차원 데카르트 좌표로 제공된 꼭짓점 `Position`을 `gl_Position`에 넣을 때

```
gl_Position = vec4(Position*0.1, 1);
```

이라는 문장을 사용. 이는 `Position`에 입력된 3차원 좌표를 10배 축소한 좌표에 동차 파라미터 1을 추가한 것

- 도형을 그리는 것은 `glDrawArrays` 함수를 사용한다.

```
void glDrawArrays(GLenum mode, GLint first, GLsizei count);
```

점을 그릴 때는 `mode`에 `GL_POINTS`라는 상수를 사용한다.

3-2. 점 그리기

선분을 그리기 위해서는 적어도 2개 이상의 점이 표현되어야 하는데, 이 역시 꼭짓점 버퍼 객체를 통해 셰이더에 전달. 다음 표 중 하나를 `mode`에 넣으면 선분을 그릴 수 있다.

에 넣을 때

```
gl_Position = vec4(Position*0.1, 1.0);
```

이라는 문장을 사용하였다(19행). 이는 Position에 입력된 3차원 좌표를 10배 축소한 좌표에 동차 파라미터 1을 추가하여 vec4형의 동차좌표를 만들어 gl_Position에 넣은 것이다.

도형을 그리는 것은 glDrawArrays 함수를 사용한다.

```
void glDrawArrays(GLenum mode, GLint first, GLsizei count);
```

이 함수에서 매개변수 mode에 그리고자 하는 도형을 나타내는 상수를 넣는데, 점을 그리기 위해서는 GL_POINTS라는 상수를 사용한다.

3.2 선분 그리기

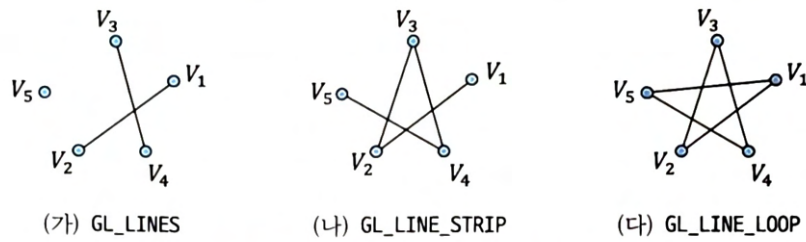
3.2.1. OpenGL 선분함수

선분을 그리기 위해서는 적어도 2개 이상의 점이 표현되어야 하는데, 이 역시 꼭짓점 버퍼 객체를 통해 셰이더에 전달한다. 선분을 그리는 것은 mode에 <표 3-1>의 상수 중 하나를 넣어 glDrawArrays 함수를 호출하면 된다.

[그림 3-2]는 V_1 부터 V_5 까지 5개의 좌표가 지정되었을 때 OpenGL 선분 그리기 상

<표 3-1> OpenGL 선분 그리기 상수

OpenGL 상수	선분의 형태
GL_LINES	꼭짓점 좌표를 2개씩 짝을 지어 선분을 그림
GL_LINE_STRIP	꼭짓점 좌표를 시작점부터 마지막 점까지 이어서 다중선을 그림
GL_LINE_LOOP	꼭짓점 좌표를 시작점부터 마지막 점까지 이은 후 다시 시작점까지 이어서 닫힌 다중선을 그림



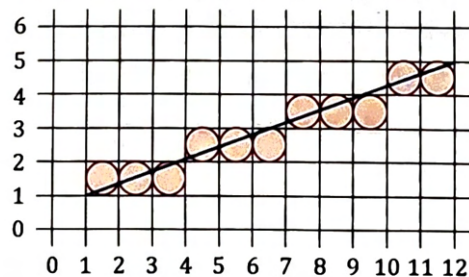
[그림 3-2] OpenGL 선분 그리기 상수

수에 의해 그린 결과이다.

3.2.2. 선분 생성 알고리즘

선분은 무수히 많은 점의 연속으로 정의된다. 그러나 래스터 그래픽스 화면에는 정수좌표만 존재하기 때문에 선분을 이루는 점 중 실수좌표는 가장 가까운 정수좌표로 근사된다. 이와 같이 선분 경로에 가장 가까운 점들의 집합으로 선분을 그리는 것을 선분의 래스터화라고 한다.

예를 들면, (5.0, 2.2)의 실수좌표는 반올림하여 (5, 2)로 근사되며, 그 결과 모니터 상의 매끄러운 선분도 확대하면 [그림 3-3]과 같이 계단 모양으로 디스플레이된다. 이러한 계단 모양은 저해상도 시스템에서 두드러지며, 선분 경로를 따라 픽셀의 밝기를 조정함으로써 부드러운 선분을 만들 수 있다.



[그림 3-3] 이상적인 선분과 실제 디스플레이되는 선분

(1) 직선 방정식

직선의 기울기-절편 방정식(slope-intercept equation)은

$$y = mx + b \quad (3-1)$$

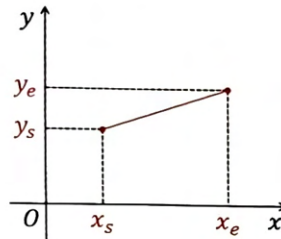
이며, m 은 직선의 기울기, b 는 y 절편이다.

선분의 두 끝점이 [그림 3-4]와 같이 (x_s, y_s) 와 (x_e, y_e) 라면 이 선분을 포함하는 직선의 기울기 m 과 y 절편 b 의 값을 식 (3-2) 및 식 (3-3)으로 계산할 수 있다.

$$m = \frac{\Delta y}{\Delta x} \quad (3-2)$$

$$b = y_s - mx_s \quad (3-3)$$

여기서 $\Delta x = x_e - x_s$, $\Delta y = y_e - y_s$ 이다.



[그림 3-4] 두 점 (x_s, y_s) 와 (x_e, y_e) 를 연결하는 선분

선분의 두 끝점 사이의 x 축 간격 Δx 와 y 축 간격 Δy 의 관계는 식 (3-2)로부터 다음과 같이 계산할 수 있다.

$$\Delta y = m\Delta x, \quad \Delta x = \frac{\Delta y}{m} \quad (3-4)$$

(2) DDA 알고리즘

디지털 차분 분석기(Digital Differential Analyzer: DDA)는 가장 간단한 선분 래스터화 알고리즘으로, 한 축의 값을 1씩 변화시킬 때 다른 축의 값이 변화하는 값을 구하여 이어지는 선분상의 픽셀 좌표를 구하는 기법이다.

기울기가 1 이하인 선분의 경우, 가장 왼쪽 끝점 $(x_0, y_0) = (x_s, y_s)$ 로부터 시작하여 x 값을 1씩 증가시키며($\Delta x = 1$) 픽셀 좌표를 구한다. k 번째 구한 x 좌표의 값 x_k 에 대하여 y_k 의 값을 계산한 뒤, 선분 경로와 가장 근접한 정숫값으로 반올림한 위치, 즉 $(x_k, \text{Round}(y_k))$ 에 픽셀을 그린다. 다음에 그릴 픽셀의 x 좌표는 $x_{k+1} = x_k + 1$ 이며, 이에 대한 y 좌표 y_{k+1} 은 다음과 같이 계산할 수 있다.

$$\begin{aligned} y_{k+1} &= mx_{k+1} + b \\ &= m(x_k + 1) + b \\ &= (mx_k + b) + m \\ &= y_k + m \end{aligned} \quad (3-5)$$

m 은 0과 1 사이의 임의의 실수가 될 수 있으므로, 각각의 계산된 y 값은 근접한 정수로 반올림된다.

기울기가 1보다 큰 양의 기울기를 갖는 선분에서는 x 와 y 의 역할을 바꾼다. 즉, y 값을 1씩 증가시켜($\Delta y = 1$) 각 y_k 값에 대하여 x_k 값을 계산한 뒤, $(\text{Round}(x_k), y_k)$ 에 픽셀을 그린다.

음의 기울기를 갖는 선분상의 픽셀 위치를 결정하기 위해서도 이와 유사한 방법으로 선분 경로상의 위치를 계산할 수 있다.

다음은 DDA 알고리즘을 구현한 프로시저이다. 이 프로시저는 선분의 끝점을 입력하고, 끝점 위치 사이의 x 좌표 및 y 좌표의 차이를 dx 와 dy 에 할당한다. 그리고 차이가 더 큰 축의 좌표를 1씩 변화시키면서 픽셀의 위치를 결정한다. 즉, x 축의 차이가 크면 $xIncrement$ 가 1 또는 -1 이 되고, y 축의 차이가 크면 $yIncrement$ 가 1 또는 -1 이 된다. (x_0, y_0) 부터 시작하여 x 축 또는 y 축으로 1만큼씩 증가(또는 감소)시키면서 선분 경로상의 다음 좌표를 구한 후, 이를 반올림하여 픽셀 위치를 생성한다.

[소스코드 3-1] DDA 알고리즘

```

1 void DDA(int x0, int y0, int xEnd, int yEnd)
2 { // x0 ≠ xEnd 또는 y0 ≠ yEnd라고 가정
3     int dx = xEnd - x0, dy = yEnd - y0, steps, k;
4     float xIncrement, yIncrement, x = x0, y = y0;

```

```

5
6     if(fabs(dx) > fabs(dy))    steps = fabs(dx);
7     else                      steps = fabs(dy);
8
9     xIncrement = float(dx) / float(steps);
10    yIncrement = float(dy) / float(steps);
11
12    setPixel(round(x), round(y));
13    for(k = 0; k < steps; k++) {
14        x += xIncrement;
15        y += yIncrement;
16        setPixel(round(x), round(y));
17    }
18 }

```

DDA 알고리즘은 증분만 더함으로써 곱셈 연산이 필요한 식 (3-1)을 직접 구현하는 것보다 빠르게 픽셀 위치를 계산하는 방법이다. 그러나 실수 연산 및 반올림 연산이 포함되어 있다. 또한 짧은 선분의 경우 큰 문제가 발생하지 않지만, 실수 연산의 오차가 누적되기 때문에 긴 선분의 경우 정확한 위치를 계산하지 못하는 단점이 있다.

(3) 브레슨햄(Bresenham)의 선분 알고리즘

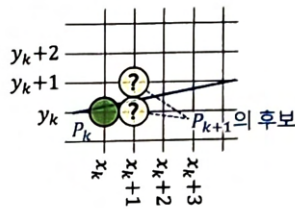
DDA 알고리즘은 x 또는 y 좌표의 변화가 실수 연산으로 계산된다는 단점이 있다. 브레슨햄의 선분 알고리즘은 실수 연산을 제거하고 정수 덧셈만으로 선분을 그릴 수 있어 DDA 알고리즘에 비해 빠르게 선분을 생성할 수 있으며, 원이나 다른 곡선에도 적용할 수 있다.

우선 기울기가 0과 1 사이인 선분을 생각해 보자. 이 경우 y 축의 증가량이 x 축의 증가량보다 작다. 선분을 구성하는 픽셀 좌표는 선분의 왼쪽 끝점 (x_0, y_0) 부터 시작하여 이어지는 픽셀 좌표를 계산한다. 선분의 기울기를 고려하면 픽셀 좌표는 x 축의 좌표를 1씩 증가시키면서 y 축의 변화를 계산하면 된다. 픽셀 좌표는 모두 정수이므로 선분에서 k 번째 픽셀 P_k 의 위치를 (x_k, y_k) 라고 표현한다면 [그림 3-5]에서처럼 $k+1$ 번째 픽셀 P_{k+1} 의 x 축 좌표는 $x_{k+1} = x_k + 1$ 이고, y 축 좌표는 $y_{k+1} = y_k$ 나

$y_{k+1} = y_k + 1$ 중 하나이다.

브레슨햄 알고리즘은 이전 픽셀의 정보로부터 다음 픽셀의 위치를 계산하는 점증적 알고리즘(incremental algorithm)의 대표적인 예이며, 곱셈을 제거하고 정수 덧셈만 사용하는 효율적인 알고리즘으로서 조금씩 다른 방법으로 구현한 여러 가지 알고리즘이 제시되고 있다.

그중 중간점 알고리즘(midpoint algorithm)으로 알려진 방법은 [그림 3-5]의 두 후보 점 사이의 중간점을 계산하여 선분이 중간점보다 위쪽을 지나가면 $y_{k+1} = y_k + 1$ 로, 선분이 중간점보다 아래쪽을 지나가면 $y_{k+1} = y_k$ 로 결정한다. 즉, 중간점의 좌표 $(x_k + 1, y_k + 0.5)$ 에서의 직선 방정식에 의해 정의되는 판별함수값을 구하고 부호를 검사하여 다음 픽셀 좌표를 결정할 수 있다.



[그림 3-5] (x_{k+1}, y_{k+1}) 픽셀의 후보 위치

기울기가 0과 1 사이라고 가정하고, 선분의 좌측 및 우측 끝점의 좌표가 각각 (x_l, y_l) , (x_r, y_r) 이라 하자. 선분 그리기의 시작점인 (x_0, y_0) 는 (x_l, y_l) 이다. 양 끝점의 수평 거리와 수직 거리는 $W = x_r - x_l$, $H = y_r - y_l$ 이며, 직선 방정식은 식 (3-6)과 같이 표현할 수 있다.

$$H(x - x_l) - W(y - y_l) = 0 \quad (3-6)$$

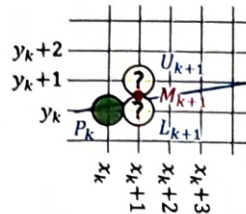
식 (3-6)의 등호를 만족하지 않는 좌표 (x, y) 의 위치는 직선의 위나 아래인데, 이를 판단하기 위해 $F(x, y)$ 를 식 (3-7)과 같이 정의한다.

$$F(x, y) = 2H(x - x_l) - 2W(y - y_l) \quad (3-7)$$

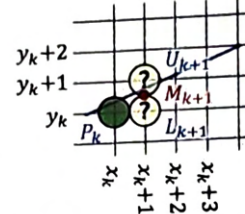
여기서 2를 곱한 것은 $F(x, y)$ 의 부호에는 영향이 없으며, 앞으로 계산식이 정수만으로

로 구성되게 하는 역할을 한다. $F(x, y) < 0$ 이면 (x, y) 는 선분의 위에, $F(x, y) > 0$ 이면 선분의 아래에 있게 된다.

선분상의 k 번째 점이 $P_k = (x_k, y_k)$ 라 할 때, 그다음 픽셀 P_{k+1} 은 [그림 3-6]과 같이 $U_{k+1} = (x_k + 1, y_k + 1)$ 과 $L_{k+1} = (x_k + 1, y_k)$ 중 하나가 된다.



(가) L_{k+1} 이 선택되는 경우



(나) U_{k+1} 이 선택되는 경우

[그림 3-6] 중간점 M_{k+1} 및 P_{k+1} 의 후보점 L_{k+1} 과 U_{k+1}

두 후보점 U_{k+1} 와 L_{k+1} 의 중간점 $M_{k+1} = (x_k + 1, y_k + 0.5)$ 에서 계산한 $F(M_{k+1})$ 은 식 (3-8)과 같다. 이때 $F(M_{k+1}) < 0$ 이면 직선이 M_{k+1} 의 아래에 있게 되므로 $P_{k+1} = L_{k+1}$ 로 결정하고, $F(M_{k+1}) > 0$ 이면 직선이 M_{k+1} 의 위에 있게 되므로 $P_{k+1} = U_{k+1}$ 로 결정한다.

$$F(M_{k+1}) = 2H(x_k + 1 - x_l) - 2W(y_k + 0.5 - y_l) \quad (3-8)$$

$k+2$ 번째 픽셀 좌표를 구하는 것은 $k+1$ 번째 점을 구할 때의 부호에 따라 결정된다. $k+1$ 번째 단계에서의 판별식 $F(M_{k+1})$ 이 음수였다면 $P_{k+1} = (x_k + 1, y_k)$ 이므로 $k+2$ 번째 단계에서 두 후보점의 중간점은 $M_{k+2} = (x_k + 2, y_k + 0.5)$ 이다. 따라서 $F(M_{k+2})$ 는 식 (3-9)와 같다.

$$\begin{aligned} F(x_k + 2, y_k + 0.5) &= 2H(x_k + 2 - x_l) - 2W(y_k + 0.5 - y_l) \\ &= F(M_{k+1}) + 2H \end{aligned} \quad (3-9)$$

$k+1$ 번째 단계에서 $F(M_{k+1})$ 이 양수였다면 $P_{k+1} = (x_k + 1, y_k + 1)$ 이므로 $k+2$ 번째 단계에서 두 후보점의 중간점은 $M_{k+2} = (x_k + 2, y_k + 1.5)$ 이다. 따라서 $F(M_{k+2})$ 는 식 (3-10)과 같다.

$$\begin{aligned}
 F(x_k + 2, y_k + 1.5) &= 2H(x_k + 2 - x_l) - 2W(y_k + 1.5 - y_l) \\
 &= F(M_{k+1}) + 2(H - W)
 \end{aligned}
 \tag{3-10}$$

식 (3-9)와 식 (3-10)에서 알 수 있듯이 판별식 $F(M_k)$ 는 이전 단계의 판별식 $F(M_{k-1})$ 에 정수 상수인 $2H$ 또는 $2(H - W)$ 만 더하면 구할 수 있다. 따라서 다음 픽셀의 위치를 결정하는 수식에서 실수 연산이 제거되므로 빠르게 선분을 그릴 수 있게 된다. 판별식의 초기값은 $F(M_1)$, $M_1 = (x_l + 1, y_l + 0.5)$ 이며, 식 (3-7)로 계산하면 $F(M_1) = 2H - W$ 이다. 다음은 브레슨햄(Bresenham)의 선분 알고리즘을 구현한 것이다.

[소스코드 3-2] Bresenham 선분 알고리즘

```

1 void bresenham_line(int xl, int yl, int xr, int yr)
2 { // 기울기가 0과 1 사이에 해당되는 경우의 Bresenham 알고리즘
3     int x, y = yl, W = xr - xl, H = yr - yl;
4     int F = 2 * H - W, dF1 = 2 * H, dF2 = 2 * (H - W);
5
6     for (x = xl; x <= xr; x++) {
7         setPixel(x, y);
8         if (F < 0)
9             F += dF1;
10        else {
11            y++;
12            F += dF2;
13        }
14    }
15 }
```

4행에서 F 의 초기값을 구한다. 또한 식 (3-9)와 (3-10)에서 F 를 업데이트하는 데 사용할 2개의 정숫값 $dF1$ 과 $dF2$ 를 미리 계산하며, 이를 9행과 12행에서 사용한다.

Bresenham 선분 알고리즘은 x - y 평면의 8분면 및 4분면의 대칭성을 이용함으로써 임의의 기울기를 갖는 선분으로 일반화될 수 있다. 기울기가 1.0보다 큰 경우 x 와 y 의 역할을 바꾼다. 음의 기울기를 갖는 경우에는 한 축의 좌표가 증가함에 따라 다른 축

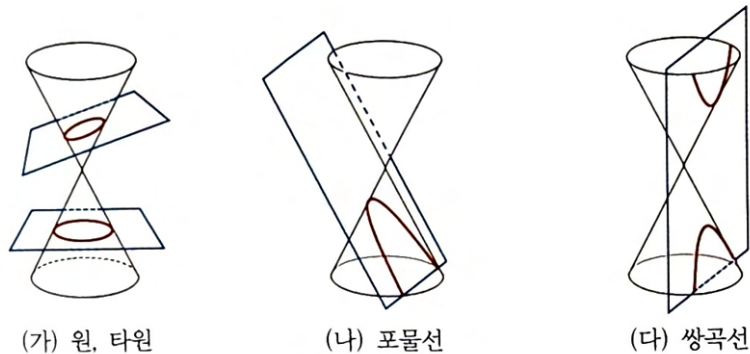
의 좌표가 감소하는 것을 제외하면 유사한 과정이다. 그리고 수평선, 수직선, 대각선의 경우에는 선분 그리기 알고리즘을 통하지 않고 간단하게 픽셀 좌표를 계산할 수 있다.

(4) 폴리라인 디스플레이하기

폴리라인(polyline)은 여러 개의 선분이 연속적으로 이어진 선을 의미한다. 폴리라인 프로시저는 n 개의 꼭짓점을 연결하는 $n-1$ 개의 선분을 디스플레이하기 위해 선분 그리기 루틴을 $n-1$ 번 호출하는 것으로 구현할 수 있다. 각각의 연속적인 호출에서는 다음 선분을 그리는 데 필요한 좌표의 쌍을 전달하는데, 이때 각 좌표쌍의 첫 번째 꼭짓점은 이전 선분의 마지막 꼭짓점이다. 첫 번째 선분을 구성하는 픽셀들의 색상값이 프레임 버퍼에 저장되고 나면 이어지는 다음 선분을 첫 번째 꼭짓점에 이어지는 다음 픽셀 위치로부터 시작하여 처리한다. 이러한 방법으로 꼭짓점의 색을 두 번 저장하는 것을 피할 수 있다.

3.3 원 그리기

[그림 3-7]과 같이 꼭짓점을 대칭으로 맞대어 놓은 직원뿔을 그 꼭짓점을 지나지 않는 평면으로 잘랐을 때 생기는 단면으로 형성되는 평면곡선을 원뿔곡선(conic section)이라고 한다. 원 및 타원, 포물선, 쌍곡선 등이 이에 해당된다.



[그림 3-7] 원뿔곡선

대표적인 원뿔곡선에 해당되는 원은 OpenGL 코어 라이브러리의 기본 함수에는 포함되어 있지 않다. 그러나 원을 많은 수의 꼭짓점을 갖는 다각형으로 근사할 수도 있으며, 타원의 특별한 경우로 취급할 수도 있다. 또는 선분의 방정식으로부터 중간점 알고리즘을 이용하여 레스터화하여 선분을 생성한 것과 유사한 방법으로 원을 생성할 수도 있다.

3.3.1. 원 생성 알고리즘

원은 그림이나 그래프에서 빈번하게 사용되는 구성요소이므로 완전한 원이나 원호를 생성하는 프로시저는 많은 그래픽스 패키지에 포함되어 있다. 그리고 종종 원과 타원을 포함한 여러 종류의 곡선을 디스플레이하기 위한 일반적인 함수가 그래픽스 라이브러리 안에 제공되기도 한다.

(1) 원의 방정식

원은 식 (3-11)과 같이 중심위치 (x_c, y_c) 로부터 반지름 r 만큼 떨어진 위치에 존재하는 점들의 집합으로 정의된다.

$$(x - x_c)^2 + (y - y_c)^2 = r^2 \quad (3-11)$$

위 식을 이용하면 x 축을 따라 $x_c - r$ 부터 $x_c + r$ 까지 단위 스텝으로 이동하면서 각 위치에서 해당되는 y 값들을 다음과 같이 계산함으로써 원주상 점들의 위치를 계산할 수 있다.

$$y = y_c \pm \sqrt{r^2 - (x_c - x)^2} \quad (3-12)$$

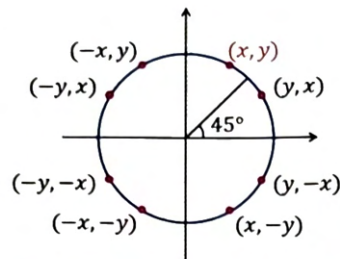
그러나 이러한 방법은 y 좌표 계산에 제곱과 제곱근 연산이 포함되기 때문에 계산량이 많다는 것과, x 축으로는 간격이 일정하지만 y 축의 간격이 일정하지 않다는 것이 문제가 된다.

이러한 문제는 극 좌표계를 이용하여 원주상의 점들을 계산하여 해결할 수 있다. 원의 방정식을 극 좌표계로 표현하면 다음의 방정식 쌍을 얻게 된다.

$$\begin{aligned} x &= x_c + r \cos \theta \\ y &= y_c + r \sin \theta \end{aligned} \quad (3-13)$$

식 (3-13)으로 고정된 각도 간격 크기를 사용하여 원을 디스플레이하면 원주를 따라 동일한 간격상의 점들로 원이 그려진다. 계산을 줄이기 위해서는 원주상의 점들 사이의 각도 간격을 크게 하고, 이 점들을 선분으로 연결하여 원의 경로를 근사화할 수 있다. 그러나 간격은 일정해지지만 역시 삼각함수 연산을 포함하고 있어 계산량이 많다.

한편, 원은 [그림 3-8]에서 보는 것처럼 원의 대칭성을 이용하면 계산을 줄일 수 있다. 즉, 원주상의 한 점만 계산하면 나머지 7개의 점의 위치는 대칭성을 이용하여 계산할 수 있다.

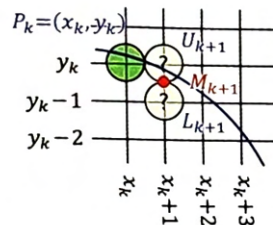


[그림 3-8] 원의 대칭성

좀 더 효율적인 방법은 Bresenham의 선분 알고리즘에서와 같이 정수연산만 포함하는 방법이다. 중간점 선분 알고리즘에서처럼 2개의 픽셀 중 어느 것을 선택할 것인지는 2개의 픽셀의 중간점에서의 함수값을 계산하여 결정한다.

(2) 중간점 원 알고리즘

래스터 선분 알고리즘에서처럼 단위 간격으로 각 스텝에서의 원 경로에 가장 가까운



[그림 3-9] 중간점 M_{k+1} 및 P_{k+1} 의 후보점 L_{k+1} 과 U_{k+1}

픽셀 위치를 결정한다. 중심이 원점에 있고 반지름이 r 인 원에 대한 판별함수를 다음과 같이 정의할 수 있다.

$$F(x, y) = x^2 + y^2 - r^2 \quad (3-14)$$

반지름이 r 인 원주상의 임의의 점 (x, y) 는 $F(x, y) = 0$ 을 만족한다. 만일 점이 원의 내부에 있다면 이 원 함수의 값은 음이고, 원 바깥의 점이라면 이 원 함수의 값은 양이다. 종합해 보면, 임의의 점 (x, y) 의 상대적 위치는 다음과 같이 원 함수의 부호를 검사하여 확인할 수 있다.

[그림 3-9]에서 $P_k = (x_k, y_k)$ 다음 픽셀은 $U_{k+1} = (x_k + 1, y_k)$ 와 $L_{k+1} = (x_k + 1, y_k - 1)$ 둘 중 하나가 된다.

중간점 $M_{k+1} = (x_k + 1, y_k - 0.5)$ 는 U_{k+1} 과 L_{k+1} 사이에 있으며, M_{k+1} 에서 식 (3-15)를 사용하여 판별식 $F(M_{k+1})$ 을 구한다. $F(M_{k+1}) < 0$ 이면 M_{k+1} 은 원의 안쪽에 있게 되므로 U_{k+1} 을 선택하고, $F(M_{k+1}) > 0$ 이면 M_{k+1} 은 원의 바깥쪽에 있게 되므로 L_{k+1} 을 선택한다.

$$F(M_{k+1}) = (x_k + 1)^2 + (y_k - 0.5)^2 - r^2 \quad (3-15)$$

$k+1$ 번째 단계에서 F 가 음수였다면(y 는 감소하지 않았음) $k+2$ 번째 단계에 대한 F 는 식 (3-16)으로 계산할 수 있다.

$$\begin{aligned} F(x_k + 2, y_k - 0.5) &= \{(x_k + 1) + 1\}^2 + (y_k - 0.5)^2 - r^2 \\ &= F(M_{k+1}) + 2x_k + 3 \end{aligned} \quad (3-16)$$

$k+1$ 번째 단계에서 F 가 양수였다면(y 는 1 감소하였음) $k+2$ 번째 단계에 대한 F 는 식 (3-17)로 계산할 수 있다.

$$\begin{aligned} F(x_k + 2, y_k - 1.5) &= \{(x_k + 1) + 1\}^2 + \{(y_k - 0.5) - 1\}^2 - r^2 \\ &= F(M_{k+1}) + 2x_k - 2y_k + 5 \end{aligned} \quad (3-17)$$

식 (3-16)과 (3-17)에서 알 수 있듯이 이전 단계의 F 에 $2x_k + 3$ 또는 $2x_k - 2y_k + 5$ 만 더하면 다음 픽셀의 위치를 결정할 수 있게 되며, 8분면의 대칭 위치에 나머지 7개의 점을 그림으로써 빠르게 원을 그릴 수 있게 된다.

다음은 중간점 원 알고리즘을 구현한 것이다.

[소스코드 3-3] 중간점 원 알고리즘

```
1 void bresenham_circle(int xc, int yc, int radius)
2 {
3     void circlePoints(int, int, int, int);
4     int F = 1 - radius;          // 중간점 파라미터의 초깃값
5     int x = 0;
6     int y = radius;
7
8     circlePoints(xc, yc, x, y);
9     while ( y >= x ) {
10         if (F < 0)                // U를 선택
11             F += x * 2 + 3;
12         else {                    // L을 선택
13             F += (x-y) * 2 + 5;
14             y--;
15         }
16         x++;
17         circlePoints(xc, yc, x, y);
18     }
19 }
20
21 void circlePoints(int xc, int yc, int x, int y)
22 {
23     setPixel(xc+x, yc+y);
24     setPixel(xc-x, yc+y);
25     setPixel(xc+x, yc-y);
26     setPixel(xc-x, yc-y);
27     setPixel(xc+y, yc+x);
28     setPixel(xc-y, yc+x);
29     setPixel(xc+y, yc-x);
30     setPixel(xc-y, yc-x);
31 }
```

4행의 판별함수값 F 의 초깃값은 $F(M_1)$, 즉 $F(1, r-0.5)$ 를 계산한 것으로 정확한 값은 $1.25-r$ 이다. 그러나 판별식의 정확한 값보다는 판별식의 부호가 중요하며, 판별식의 변화는 정숫값만큼씩 발생하므로 r 이 정수인 경우는 0.25를 제거한 $1-r$ 을 사용함으로써 정수연산만 포함되도록 구현하고 있다.

3.3.2. 다른 곡선

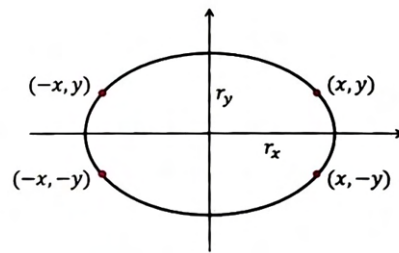
타원, 포물선, 쌍곡선 등의 곡선은 객체를 모델링하거나 궤도 운동 및 움직임을 표현할 때 유용하게 사용된다. 곡선을 그리는 단순한 방법은 많은 꼭짓점을 갖는 다각형으로 근사하는 방법이 있다. 또 다른 방법으로는 $y=f(x)$ 또는 $f(x, y)=0$ 의 형태로 주어진 곡선의 함수식으로부터 점증적 중간점 방법을 사용하여 좌표를 계산하고 래스터화하여 곡선을 그릴 수 있다.

다음에 몇 가지 유용한 곡선의 방정식을 소개한다.

(1) 타원

타원은 원의 일반형이라고 볼 수 있다. [그림 3-10]과 같이 장축과 단축이 좌표축에 정렬된 방향으로 놓여 있다면 타원 방정식은 매우 단순한 형태가 된다. 타원 방정식은 타원의 중심 좌표와 파라미터 r_x , r_y 의 항으로 다음과 같이 쓸 수 있다.

$$\left(\frac{x-x_c}{r_x}\right)^2 + \left(\frac{y-y_c}{r_y}\right)^2 = 1 \quad (3-18)$$



[그림 3-10] 타원과 대칭성

극좌표 r 과 θ 를 사용하여 표준 위치의 타원 방정식을 식 (3-19)와 같이 표현할 수도 있다.

$$\begin{aligned} x &= x_c + r_x \cos \theta \\ y &= y_c + r_y \sin \theta \end{aligned} \quad (3-19)$$

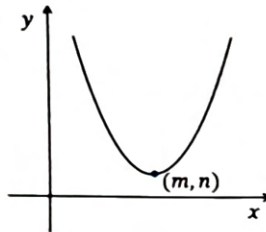
타원의 편심각(eccentric angle)이라고 하는 각 θ 는 경계 원의 원주를 돌면서 측정된다. 만일 $r_x > r_y$ 이면 경계 원의 반지름이 $r = r_x$ 이고, 그렇지 않으면 경계 원의 반지름이 $r = r_y$ 이다. 원의 경우와 같이 대칭성을 이용하여 계산량을 줄일 수 있다. 단, 원의 경우는 8분면에 관하여 대칭점을 찾을 수 있으나, 타원의 경우에는 4분면에서의 대칭점만 찾을 수 있다.

(2) 포물선

포물선의 그래프는 2차 함수로 주어지고, 일반 방정식은 식 (3-20)과 같으며, [그림 3-11]과 같이 꼭짓점의 좌표가 (m, n) 인 포물선의 방정식은 식 (3-21)과 같다.

$$y = ax^2 + bx + c \quad (3-20)$$

$$y = a(x - m)^2 + n \quad (3-21)$$



[그림 3-11] 꼭짓점이 (m, n) 인 포물선

(3) 쌍곡선

원점에 중심을 둔 쌍곡선의 일반 방정식은 식 (3-22)와 같다.

$$\left(\frac{x}{r_x}\right)^2 - \left(\frac{y}{r_y}\right)^2 = 1 \quad (3-22)$$

이 방정식은 타원 방정식과 비교하였을 때 x^2 항과 y^2 항 사이의 부호만 다르므로 타원 알고리즘을 약간 수정하여 쌍곡선 경로상의 점들을 발생시킬 수 있다.

(4) 원뿔곡선

원뿔곡선은 식 (3-23)과 같은 2차 방정식으로 표현될 수 있다.

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0 \quad (3-23)$$

여기서 파라미터 A, B, C, D, E, F 는 디스플레이하려는 곡선의 종류를 결정한다. 곡선의 종류는 식 (3-24)와 같이 $B^2 - 4AC$ 의 값에 따라 판별할 수 있다.

$$B^2 - 4AC \begin{cases} < 0, \text{ 타원 (또는 원)을 생성} \\ = 0, \text{ 포물선을 생성} \\ > 0, \text{ 쌍곡선을 생성} \end{cases} \quad (3-24)$$

예를 들어, $A = C = 1, B = 0, D = -2x_c, E = -2y_c, F = x_c^2 + y_c^2 - r^2$ 이면 원의 방정식이다.

3.4 다각형 그리기

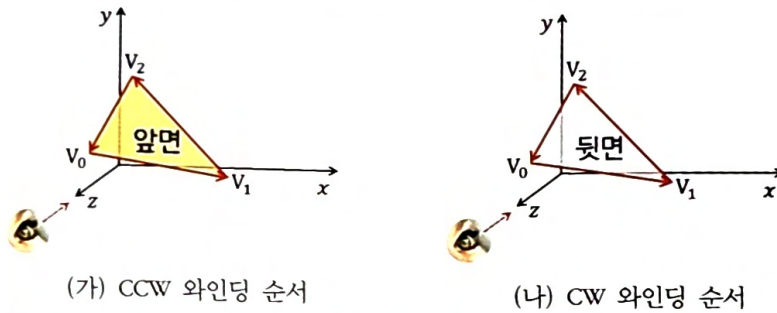
3.4.1. 다각형

다각형(polygon)의 수학적 정의는 다각형의 변(sides) 또는 모서리(edges)라고 하는 선분에 의해 순차적으로 연결된 꼭짓점(vertices)이라고 하는 3개 이상의 좌표 위치의 집합으로 지정된 평면도형이다. 특히 다각형을 구성하는 변이 서로 교차하지 않는다면 그 다각형은 단순다각형(simple polygon)이라고 한다.

컴퓨터 그래픽스에서 다각형은 단순히 선분의 연속이 아니라 선분에 둘러싸인 면을 포함하는 의미를 갖고 있다. 특히 입체적인 물체를 표현할 때 물체의 표면을 작은 다각형의 조합으로 근사화하여 표현할 수 있어서 다각형은 중요한 의미를 갖는다.

다각형을 그릴 때, 다각형의 앞면을 정하기 위해 꼭짓점을 나열하는 순서를 와인딩 순서(winding order)라고 한다. 다각형의 앞면을 바라볼 때 꼭짓점을 시계 반대방향(CounterClockWise: CCW)이 되도록 나열한다면, 시계 반대방향 와인딩 순서라고 한다. 예를 들어, [그림 3-12]에서 V_0, V_1, V_2 의 순서로 나열하여 삼각형을 정의하였다면, CCW 순서로 나열된 것이므로 보이는 면이 앞면이다. 만일 시계 방향(ClockWise: CW) 와인딩 순서를 적용하는 것으로 하였다면, 꼭짓점을 V_0, V_2, V_1 의 순서로 나열하여 삼각형을 정의하였을 때 앞면이 보이는 것으로 판단한다.

OpenGL에서는 특별히 지정하지 않으면 다각형의 앞면에 대하여 시계 반대방향의 와인딩 순서가 적용된 것으로 간주한다. 따라서 시계 방향으로 나열된 것으로 보이는 면은 뒷면으로 간주한다. 앞면과 뒷면을 구분하는 이유는 뒷면을 보이지 않게 하거나 앞면과 뒷면의 색상을 다르게 부여할 수 있어야 하기 때문이다.



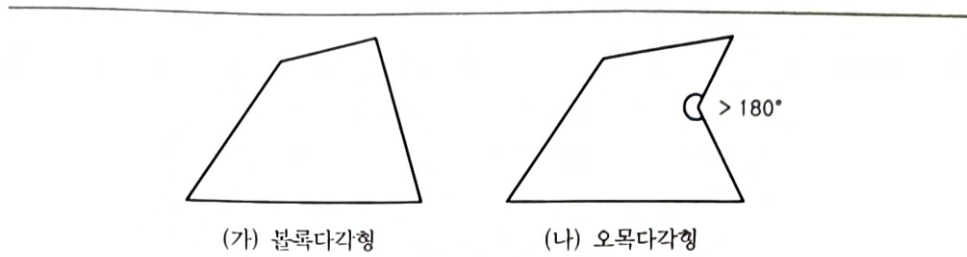
[그림 3-12] 와인딩 순서 및 다각형의 앞면·뒷면(꼭짓점을 V_0, V_1, V_2 순서로 나열한 경우)

2차원 공간에서 단순다각형의 면적은 신발끈 공식(shoelace formula)으로 구할 수 있다. 만일 다각형의 꼭짓점 $(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})$ 이 CCW 순서로 나열된 것이라면, 다각형의 면적은 식 (3-25)로 구할 수 있다.

$$area = \frac{1}{2} \sum_{i=0}^{n-1} (x_i y_{(i+1) \bmod n} - x_{(i+1) \bmod n} y_i) \quad (3-25)$$

만일 꼭짓점이 CW 순서로 나열되었다면 식 (3-25)의 면적은 음수로 계산될 것이며, 이러한 특성은 다각형의 앞면과 뒷면을 판별하는 데 활용할 수 있다.

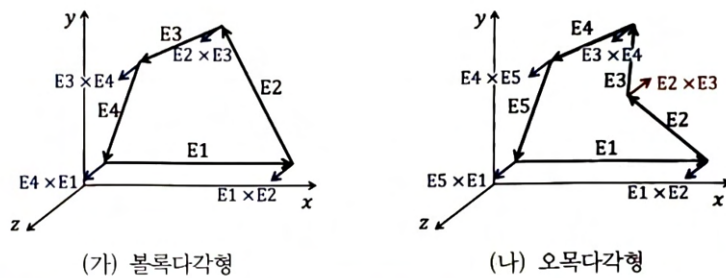
다각형의 내각은 인접한 두 변으로 형성되는 다각형 경계 내부의 각이다. 만일 다각형



[그림 3-13] 볼록다각형과 오목다각형

의 모든 내각이 180° 보다 작으면 볼록(convex)다각형이라고 한다. 만일 1개 이상의 꼭짓점 내각이 180° 보다 크면 오목(concave)다각형이다(그림 3-13 참조).

다각형이 볼록한지 검사하는 방법으로는 인접한 꼭짓점으로 향하는 벡터 쌍의 외적을 이용하는 방법이 있다. 다각형의 꼭짓점을 와인딩 순서에 따라 연결하는 벡터로 표현한 후, 와인딩 순서로 인접한 두 벡터의 외적을 구하였을 때 모든 외적이 같은 방향을 가리킨다면 볼록다각형, 그렇지 않으면 오목다각형이다(그림 3-14 참조).



[그림 3-14] 꼭짓점을 연결하는 벡터 쌍의 외적을 이용한 볼록다각형·오목다각형 판별

오목다각형을 검사하는 다른 방법으로는 임의의 변의 연장선에 대한 다각형의 꼭짓점 위치를 살펴보는 방법이 있다. 만약 어떤 꼭짓점은 연장선의 어느 한쪽에 있고, 어떤 꼭짓점은 다른 한쪽에 있다면 그 다각형은 오목하다.

3.4.2. 다각형의 내부·외부 판별

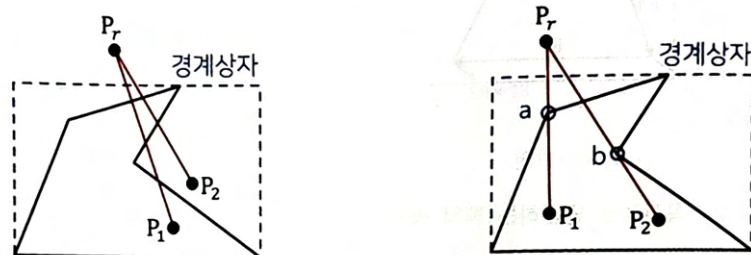
다양한 그래픽 처리에서 종종 객체 영역의 내부를 식별해야 한다. 볼록다각형, 원,

구와 같은 단순한 객체의 내부를 식별하는 것은 일반적으로 간단한 작업이다.

평면도형의 내부 영역을 식별하기 위해 사용되는 알고리즘으로는 홀-짝 규칙(odd-even rule)이 있다. 이 규칙은 홀수 패리티 규칙(odd-parity rule) 또는 짝-홀 규칙(even rule)이라고도 한다. 먼저 임의의 위치 P 로부터 닫힌 다중선의 좌표범위 외부의 점 P_r 까지의 직선을 가상으로 그린다. 그리고 나서 이 판별선을 따라 교차하는 변의 수를 센다. 만일 이 판별선과 교차하는 변의 수가 홀수이면 P 는 내부 점으로 간주된다. 그렇지 않으면 P 는 외부 점이다. 그런데 이 판별선이 꼭짓점을 교차하는 경우는 홀 규칙을 단순히 적용하는 것으로는 정확한 판별을 할 수 없다.

[그림 3-15]의 (가)에서는 판별선이 꼭짓점과 교차하지 않는다. 이 경우 홀-짝 규칙에 따라 P_1 은 교차횟수가 홀수(3회)이므로 다각형 내부의 점이고, P_2 는 교차횟수가 짝수(2회)이므로 외부의 점이다.

[그림 3-15]의 (나)는 판별선이 꼭짓점과 교차하는 경우의 예이다. 꼭짓점을 공유하는 두 변과 모두 교차하는 것으로 계산하거나 모두 교차하지 않는다고 계산한다면 P_1 은 교차횟수가 짝수가 되어 외부로 판단하는 오류가 발생한다. 반면에, 꼭짓점은 1회의 교차로 계산한다면 P_2 가 외부인 것으로 오판하게 된다.



(가) 판별선이 꼭짓점과 교차하지 않는 경우

(나) 판별선이 꼭짓점과 교차하는 경우

[그림 3-15] 홀-짝 규칙에 따른 다각형의 내부·외부 판별

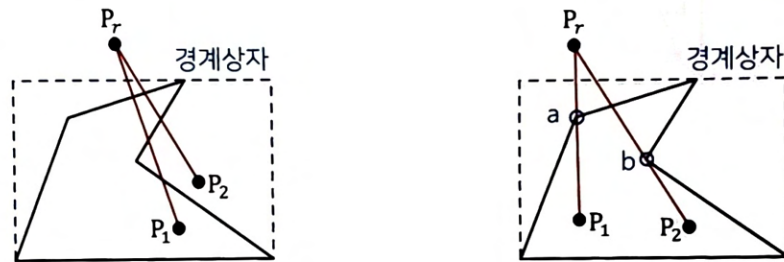
판별선이 꼭짓점과 교차하는 경우는 꼭짓점을 공유하는 두 변이 판별선을 기준으로 양쪽에 나뉘어 배치되어 있는지, 아니면 한쪽에 모여 있는지에 따라 판단하는 것으로 처리할 수 있다. [그림 3-16]의 (가)와 같이 양쪽에 나뉘어 배치된 경우에는 그중 하나만 교차하는 것으로, (나)나 (다)와 같이 어느 한쪽에 모여 있는 경우는 모두 교차한다.

구와 같은 단순한 객체의 내부를 식별하는 것은 일반적으로 간단한 작업이다.

평면도형의 내부 영역을 식별하기 위해 사용되는 알고리즘으로는 **홀-짝 규칙**(odd-even rule)이 있다. 이 규칙은 홀수 패리티 규칙(odd-parity rule) 또는 짝-홀 규칙(even-odd rule)이라고도 한다. 먼저 임의의 위치 P 로부터 닫힌 다중선의 좌표범위 외부의 기준점 P_r 까지의 직선을 가상으로 그린다. 그리고 나서 이 판별선을 따라 교차하는 변의 수를 센다. 만일 이 판별선과 교차하는 변의 수가 홀수이면 P 는 내부 점으로 간주된다. 그렇지 않으면 P 는 외부 점이다. 그런데 이 판별선이 꼭짓점을 교차하는 경우는 홀-짝 규칙을 단순히 적용하는 것으로는 정확한 판별을 할 수 없다.

[그림 3-15]의 (가)에서는 판별선이 꼭짓점과 교차하지 않는다. 이 경우 홀-짝 규칙에 따라 P_1 은 교차횟수가 홀수(3회)이므로 다각형 내부의 점이고, P_2 는 교차횟수가 짝수(2회)이므로 외부의 점이다.

[그림 3-15]의 (나)는 판별선이 꼭짓점과 교차하는 경우의 예이다. 꼭짓점을 공유하는 두 변과 모두 교차하는 것으로 계산하거나 모두 교차하지 않는다고 계산한다면 P_1 은 교차횟수가 짝수가 되어 외부로 판단하는 오류가 발생한다. 반면에, 꼭짓점은 1회의 교차로 계산한다면 P_2 가 외부인 것으로 오판하게 된다.



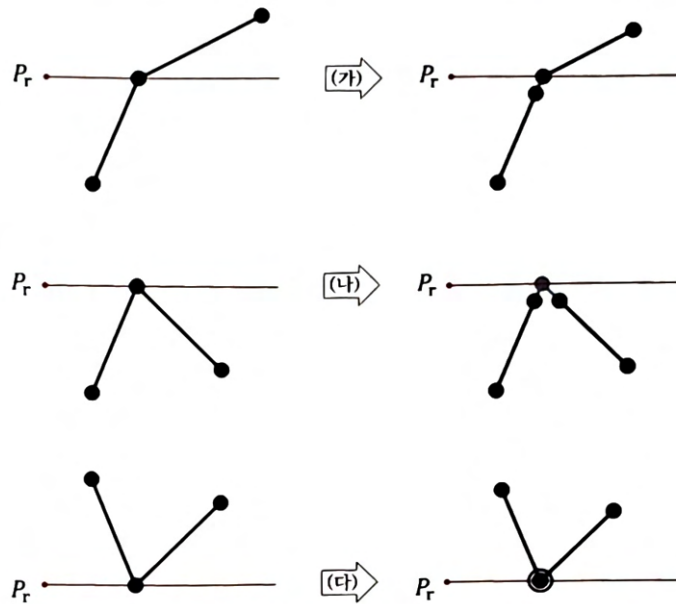
(가) 판별선이 꼭짓점과 교차하지 않는 경우

(나) 판별선이 꼭짓점과 교차하는 경우

[그림 3-15] 홀-짝 규칙에 따른 다각형의 내부·외부 판별

판별선이 꼭짓점과 교차하는 경우는 꼭짓점을 공유하는 두 변이 판별선을 기준으로 양쪽에 나뉘어 배치되어 있는지, 아니면 한쪽에 모여 있는지에 따라 판단하는 것으로 처리할 수 있다. [그림 3-16]의 (가)와 같이 양쪽에 나뉘어 배치된 경우에는 그중 하나만 교차하는 것으로, (나)나 (다)와 같이 어느 한쪽에 모여 있는 경우는 모두 교차하게

나 모두 교차하지 않는 것으로 처리하는 것이다. 이러한 처리는 판별선을 x 축과 평행한 선으로 하고, 변의 두 꼭짓점 중 y 좌표가 큰 꼭짓점은 선분에 포함되지 않는 것으로 계산하여 해결할 수 있다. 만일 x 축과 평행한 변이 있다면 그 변은 계산에서 제외한다.



[그림 3-16] 판별선이 꼭짓점을 통과하는 경우의 출-짝 규칙 처리

3.4.3. 다각형의 앞면과 뒷면의 판별

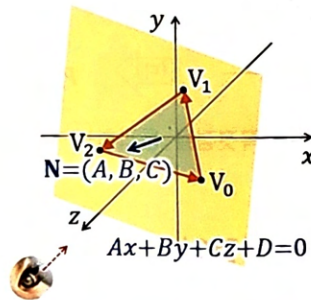
우리는 일반적으로 객체 내부를 싸고 있는 다각형 표면을 다루기 때문에 각 표면의 양쪽 면을 구분해야 한다. 객체의 내부를 향하는 다각형 면을 뒷면(back face)이라고 하며, 보이는 쪽, 즉 바깥쪽을 향하는 면을 앞면(front face)이라고 한다. 공간상에서 어떤 점이 다각형의 앞면과 뒷면을 기준으로 어느 방향에 위치하는지 알아내는 것은 객체의 가시성을 결정하는 예와 같은 많은 그래픽 알고리즘에서 기본적인 작업이다. 모든 다각형은 공간을 두 영역으로 분할하는 무한 평면 내에 포함되어 있다. 평면 위에 있지 않고 다각형 평면의 앞면에서 볼 수 있는 임의의 점은 평면의 앞(또는 외부)이라고 하

며, 따라서 객체의 바깥이 된다. 그리고 다각형의 뒷면에서 볼 수 있는 임의의 점은 평면의 뒤(또는 내부)이다. 잊지 말아야 할 것은 이러한 내부·외부 분류는 다각형을 포함하고 있는 평면에 상대적인 것인 반면, 앞서 논한 홀·짝 규칙을 이용한 내부·외부 검사는 2차원 경계의 내부에 관한 것이다.

3차원 공간에서 평면의 방향은 그 면에 수직인 벡터로 표현할 수 있는데, 이를 그 평면의 법선 벡터(normal vector)라고 한다. 벡터 N 이 어떠한 평면에 수직인 법선 벡터라면 $-N$ 역시 수직인 벡터이다. 그중 벡터 N 을 앞면이 향하는 벡터로 정한다면, $-N$ 은 뒷면이 향하는 벡터이다. 평면의 앞과 뒤는 상대적인 것이므로 적절히 정한 규칙에 따라 다각형의 앞면과 뒷면을 정의한다.

와인딩 순서가 CCW일 경우 [그림 3-17]에서 꼭짓점 V_0, V_1, V_2 에 의해 정의되는 삼각형의 앞면 방향을 나타내는 법선 벡터 N 은 와인딩 순서에 따라 꼭짓점을 연결하는 벡터의 외적으로 구할 수 있다. 즉, $E_1 = V_1 - V_0$ 와 $E_2 = V_2 - V_1$ 이라 할 때 법선 벡터 $N = (A, B, C)$ 는 식 (3-26)으로 구할 수 있다.

$$N = E_1 \times E_2 \quad (3-26)$$



[그림 3-17] 다각형의 법선 벡터와 앞면의 방향

평면 방정식은 다각형을 포함하는 평면에 대한 공간 내 점들의 상대적 위치를 정하는 데 사용될 수 있다. [그림 3-17]에서 삼각형을 포함하는 평면 방정식은 식 (3-26)으로 구한 법선 벡터 $N = (A, B, C)$ 에 따라 식 (3-27)로 표현할 수 있다.

$$Ax + By + Cz + D = 0 \quad (3-27)$$

이때, $D = -(Ax + By + Cz)$ 이므로 입의의 꼭짓점 좌표를 이용하여 D 를 구할 수 있다.

평면에 존재하지 않는 점 (x, y, z) 가 이 평면의 앞에 있는지, 아니면 뒤에 있는지 $Ax + By + Cz + D$ 의 부호(양 또는 음)에 따라 다음과 같이 판정할 수 있다.

만일 $Ax + By + Cz + D > 0$ 이면 점 (x, y, z) 는 평면의 앞에 있다.

만일 $Ax + By + Cz + D < 0$ 이면 점 (x, y, z) 는 평면의 뒤에 있다.

[그림 3-17]과 같이 시선의 방향이 $-z$ 축 방향일 때 관찰자에게 보이는 다각형의 면이 앞면인지 아니면 뒷면인지를 판단하는 것은 다각형 법선 벡터의 z 축 성분인 C 의 부호로 판단할 수 있다. 만일 $C > 0$ 이라면 다각형의 앞면이 관찰자에게 보이는 것이다. 예를 들어, 삼각형의 꼭짓점 좌표가 $V_0 = (1, 2, 3)$, $V_1 = (5, 1, 2)$, $V_2 = (2, 4, 1)$ 이라면 식 (3-26)의 E_1 과 E_2 는 각각 $(4, -1, -1)$ 과 $(-3, 3, -1)$ 이며, 법선 벡터 N 은 $(4, 7, 9)$ 이다. 이때, N 의 z 축 성분값인 9가 0보다 크므로 관찰자에게 이 삼각형의 앞면이 보이는 것으로 판단할 수 있다.

식 (3-25)의 신발끈 공식을 이용하여 다각형의 앞면·뒷면을 식별할 수도 있다. CCW 와인딩 순서를 적용하는 경우, 다각형을 $x-y$ 평면에 수직으로 투영(z 축의 좌표가 0이 됨)한 면적을 식 (3-25)로 구하였을 때 그 값이 양이면 앞면, 음이면 뒷면으로 판단한다. 앞의 예에서 V_0, V_1, V_2 를 $x-y$ 평면에 수직으로 투영한 후 식 (3-25)로 구한 값은 4.5이며, 이는 관찰자가 이 삼각형의 앞면을 보고 있음을 의미한다.

3.4.4. 다각형 표

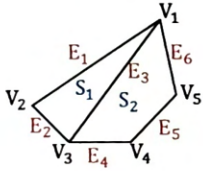
장면 내의 객체들은 다각형 면들의 집합으로 묘사된다. 실제로 그래픽스 패키지는 입체의 표면을 다각형 조각들의 그물 구조로 정의하는 함수를 제공한다. 각 객체의 묘사에는 다각형 면들의 기하구조를 지정하는 좌표 정보와 색, 투명도, 조명-반사 특성과 같은 기타 표면 파라미터 등을 포함한다. 각 다각형에 대한 정보가 입력되면, 이 데이터를 후속처리, 디스플레이, 장면 내에서의 객체 조작 등에 사용될 수 있도록 표에 저장한다. 이러한 다각형 표는 기하구조 데이터 표와 속성 표의 두 그룹으로 구성될 수 있다. 기하구조 데이터 표는 꼭짓점 좌표와 다각형 면의 공간적 방향을 나타내는 파라미터 등을 담고 있다. 객체에 대한 속성 정보에는 객체의 투명도, 객체 표면의 반

사율과 텍스처 특성 등을 지정하는 파라미터가 포함된다.

장면 내의 객체들에 대한 기하구조 데이터는 꼭짓점 표, 변 표, 패싯(surface-facet: 다면체의 면을 구성하는 다각형) 표라는 3개의 리스트에 배치된다. 꼭짓점 표에는 객체의 꼭짓점들의 좌표값이 저장된다. 변 표에는 다각형을 구성하는 변의 꼭짓점 정보를 알아낼 수 있도록 꼭짓점 표의 해당 항목에 대한 포인터를 담고 있다. 각 변에는 그 변을 포함하는 패싯에 대한 포인터를 가지고 있게 하여 그 변을 공유하는 인접 패싯을 쉽게 판별하게 할 수도 있다. 그리고 패싯 표에는 다각형을 구성하는 변을 알 수 있도록 변 표의 항목에 대한 포인터를 저장한다. [그림 3-18]은 다각형 표의 예를 보여 준다.

꼭짓점 표와 패싯 표만 사용하여 기하구조 데이터를 표현할 수도 있다. 그러나 이 방법은 인접한 패싯 쌍이 공유하는 변을 식별하는 것이 직관적이지 않으며, 와이어 프레임 디스플레이를 할 때 일부 변이 두 번 그려질 수도 있다.

패싯 표만 사용하여 기하구조 데이터를 표현하는 것도 가능하다. 그러나 이 방법은 각 패싯 단위로 꼭짓점의 좌표값을 나열하기 때문에 좌표 정보를 중복하여 저장하게 된다. 또한 패싯 표 안의 꼭짓점 목록으로부터 다른 패싯과의 관계를 다시 구성해 내야 한다.



꼭짓점 표	변 표	패싯 표
$V_1: x_1, y_1, z_1$	$E_1: V_1, V_2, S_1$	$S_1: E_1, E_2, E_3$
$V_2: x_2, y_2, z_2$	$E_2: V_2, V_3, S_1$	$S_2: E_3, E_4, E_5, E_6$
$V_3: x_3, y_3, z_3$	$E_3: V_3, V_1, S_1, S_2$	
$V_4: x_4, y_4, z_4$	$E_4: V_3, V_4, S_2$	
$V_5: x_5, y_5, z_5$	$E_5: V_4, V_5, S_2$	
	$E_6: V_5, V_1, S_2$	

[그림 3-18] 다각형 표

3.4.5. OpenGL 삼각형 그리기 함수

OpenGL의 코어 프로파일에서는 일반적인 다각형은 지원하지 않고 삼각형만 제공한다. 삼각형은 세 꼭짓점이 한 평면상에 있어야 한다는 조건이 무조건 만족되지만, 그

외의 다각형을 그리기 위한 꼭짓점은 실수 표현의 오차로 인해 그렇지 않을 수도 있다. 또한 모든 삼각형은 볼록다각형이라는 특징이 있다는 점도 고려하였다. 왜냐하면 오목다각형은 제어하는 과정이 복잡하며, 효율적인 렌더링 알고리즘을 사용할 수 없기 때문이다. 만일 임의의 다각형을 표현하고자 한다면 2개 이상의 삼각형을 결합하여 표현해야 한다. 삼각형을 그리는 것은 mode에 `GL_TRIANGLES`, `GL_TRIANGLE_STRIP`, `GL_TRIANGLE_FAN`이라는 상수 중 하나를 넣어 `glDrawArrays` 함수를 호출하면 된다.

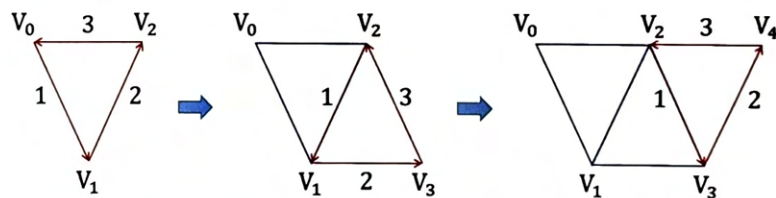
(1) 삼각형

삼각형은 가장 기본적인 다각형으로 3개의 변을 가진 도형을 말한다. `GL_TRIANGLES` 기본 모델은 3개의 꼭짓점을 연결하여 삼각형을 생성한다. 만일 N 개의 꼭짓점으로 구성된 리스트에서 반복되는 좌표 위치가 없다고 가정하면 $N/3$ 개의 삼각형을 얻게 된다. 물론 $N \geq 3$ 이어야 하며, 그렇지 않으면 아무것도 디스플레이되지 않는다.

(2) 삼각형 스트립

삼각형을 여러 개 연결하면 다양한 면과 모양의 다각형을 그릴 수 있다. 선을 연결할 때와 마찬가지로 `GL_TRIANGLE_STRIP` 상수를 이용하면 삼각형을 연결하여 그릴 수 있다. V_0, V_1, V_2, V_3, V_4 의 순서로 나열된 꼭짓점에 이 상수가 지정되었을 때 삼각형 스트립이 그려지는 순서는 [그림 3-19]와 같다. 처음 세 꼭짓점 V_0, V_1, V_2 가 시계 반대방향으로 나열되어 삼각형의 앞면 방향을 결정한다. 나머지 삼각형은 꼭짓점이 지정된 순서대로 그려지는 것이 아니라 시계 반대방향의 와인딩 순서를 유지하는 방식으로 그려진다.

스트립을 이용하여 다각형을 생성하면 별도의 삼각형을 그리지 않고 삼각형에 하나의 꼭짓점을 추가함으로써 삼각형을 그릴 수 있기 때문에, 데이터 공간을 절약할 수

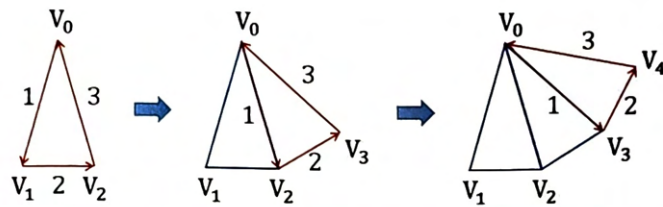


[그림 3-19] `GL_TRIANGLE_STRIP`으로 생성한 도형

있을 뿐 아니라 실행속도도 향상시킬 수 있다.

(3) 삼각형 팬

GL_TRIANGLE_FAN 상수를 사용하면 첫 번째 지정된 꼭짓점을 중심으로 부채꼴 모양의 연결된 삼각형을 만들 수 있다. GL_TRIANGLE_STRIP과 마찬가지로 처음 삼각형 다음에 꼭짓점 하나만 추가하면 연속하여 삼각형을 생성할 수 있다. [그림 3-20]은 V_0 , V_1 , V_2 , V_3 , V_4 의 순서로 나열된 꼭짓점에 이 상수가 지정되었을 때 삼각형이 그려지는 순서이다. 처음 3개의 꼭짓점이 시계 반대방향으로 나열되어 삼각형의 앞면 방향을 결정하며, 나머지 꼭짓점으로 삼각형 앞면 방향을 유지하며 삼각형을 연결하여 그린다.



[그림 3-20] TRIANGLE_FAN으로 생성한 도형